Judged Placement in a Live Deliberation-Synthesis Pipeline

How Hebrew clustering accuracy rose from 0.066 to 0.932, what each fix was, and what it costs in latency and money

Freedi live-synthesis research series · 2026-08-21 · Study root: `scientific-research/2026-08-18-live-synth-accuracy/` · Prepared by Claude (Anthropic) for Tal Yaron · All runs certified against build fingerprints; nothing in this report is deployed.

Abstract. Freedi's live synthesis pipeline clusters community proposals in real time: near-duplicate statements are merged into a single synthesized proposal, and syntheses are filed under topic headings. A 100-statement benchmark with known ground truth (10 topics × 5 ideas × 2 paraphrases) scores the pipeline as $0.6 \cdot F1_{synth} + 0.4 \cdot F1_{topic}$. This report covers the third phase of the study (2026-08-20 → 08-21): diagnosing why the theme layer stalled below its ceiling in English, building per-question embedding-model migration for Hebrew, and closing the gap Hebrew's harder vector geometry opened. The result is **English 0.902** (best certified band 0.884–0.910) and **Hebrew 0.932** — the highest score of any certified run in the study, with synthesis precision 1.000 and zero false merges. Every gain traces to one law applied repeatedly: *embedding distance may propose; only an LLM judge may decide*. The changes leave per-statement wall-clock time unchanged in English (33.8 s → 33.8 s under identical harness pacing) and add roughly one fast-tier LLM call per statement, an increase of ≈8–15% in per-statement LLM cost, concentrated in the cheap model tier. Migrating one Hebrew question is a single administrative call.

1 Background

The pipeline under test is `runSinglePipeline`, fired for every created option (`liveSynthOnOptionCreate`). It decides, per statement: attach to an existing synthesis, spawn a new synthesis with a partner, file under a topic heading, queue for review, or wait. Earlier phases of this study took English from 0.067 (a single topic cluster swallowed all 100 statements) to a certified band of 0.884–0.910 across three arrival-order seeds, with the synthesis layer exact (150/150 pairs, zero false merges). Hebrew, at that point, scored 0.066 at defaults and 0.369 with a hand-tuned configuration — the embedding model (`text-embedding-3-small`) simply cannot see Hebrew paraphrases (true twin is nearest neighbour: EN 100/100, HE 56/100).

Two work orders drove this phase: (a) continue improving theming, and (b) execute the decided Hebrew rollout — per-question migration to `text-embedding-3-large` — and push Hebrew toward ≈0.9.

2 Methods

2.1 The benchmark and its certification protocol

Each run feeds 100 statements (frozen corpus, seeded shuffle) one at a time into Firebase emulators running the compiled functions build, waits for the pipeline to settle per statement, and exports the resulting cluster structure plus a full audit trail. A run is *certified* only if: the scorer's self-test passes; the `functions/lib` build fingerprint (SHA over all compiled files) is byte-identical before and after; and no rebuild occurred during the run window (newest artifact mtime precedes run start). Uncertified runs are reported but never cited.

2.2 Instruments built for this phase

Instrument	What it measures
`dumpEmulatorEvidence.mjs`	Rescues the full audit log (with before/after membership snapshots the exporter drops) and all statement documents from a live emulator.
`themeFiling.mjs`	Reconstructs the exact theme set at every filing decision of a run (certified byte-for-byte against the live sweep fingerprint) and re-asks each decision under prompt variants. The instrument behind Findings 15–16.
`heBands.mjs`	The three cosine distributions (twin / same-topic / cross-topic) of the Hebrew corpus under 3-large embeddings, with operating characteristics per candidate threshold.
`spawnJudgeAB.mjs`	A/B bench for the synthesis judge itself: 50 true twins (must merge) + 10 hardest same-topic negatives (must refuse), per language, driven through the *compiled* function.

2.3 Discipline

Three rules were enforced throughout, each purchased by an earlier mistake in this study. (1) *Never re-implement code under test* — every offline A/B drives the compiled artifact (Finding 8). (2) *Replay on the states the system actually meets*, not on tidy end states (Finding 13, which mispredicted a live regression from an end-state replay). (3) *Pre-register decision criteria* before results arrive — the filing-prompt selection rule and the spawn-judge fix criteria were both fixed in writing before the deciding measurements ran.

3 What was fixed

Eight interventions shipped in this phase, each measured individually. Commits in parentheses.

1. **Filing diagnosis and fix** (`df336521c`, `d576f27f4`). Rebuilding a regressed run's exact mid-run states proved theme impurity enters at *filing* (the judge that picks a heading for each new proposal), not at the consolidation sweep previously blamed — live filing accuracy was 62%, with every misfile landing in a broad-titled "attractor" heading. The fix: the filing judge now sees each heading's actual *contents*, and its doubt-bias is flipped to "when unsure, answer NONE" (a needless new heading self-heals — the consolidation sweep merges duplicates, measured 12/13 clean — while a misfile is permanent). Only the combination works: contents alone made misfiling *worse* (28→37 per 174 replayed decisions); together, 28→17. Live confirmation: misfiles 12→5, English composite back in band at 0.902.
2. **Per-question embedding model** (`5a8d37692`). A question can pin `text-embedding-3-large` while the rest of the corpus stays on 3-small. Generation, the cache's compatibility guard, and vector search all resolve the model per question; mixed-model cosines (numbers with no

meaning) are structurally impossible. Migration is one call: `reEmbedQuestion({questionId, embeddingModel})`.

3. **Per-model cosine bands** (`e3a240562`). The pipeline's thresholds are calibrated to a geometry, not to the pipeline. Measured on Hebrew 3-large: 4,421 of 4,500 cross-topic pairs cleared the old topic band, and 234 wrong pairs sat inside the old spawn band. A pinned question now gets bands measured for its space (attach 0.86 / spawn 0.80 / candidacy 0.75), written automatically at migration.
4. **Topic attach is judged** (`e3a240562`). The one placement path that attached by cosine alone (it built a 45-member cross-topic mega-theme in Hebrew, 25 of 45 members entering unjudged) is disabled whenever LLM theme assignment is on; the filing judge decides every placement.
5. **Spawn retry over a shortlist** (`bfff1c00b`). A merge-refusal is a verdict on one pair, not on the statement: the judge now sees up to 3 in-band partner candidates. In compressed Hebrew space the true twin ranked below a wrong neighbour, and the right pair was never previously offered.
6. **Wider neighbour window** (`bfff1c00b`). 10→15, because ~60 cluster-title vectors crowd a 10-wide window when the whole space packs into 0.64–0.94 (a true twin ranked #10 behind four theme titles).
7. **Formulation ≠ intervention** (`97e3b57dc`). The synthesis judge refused Hebrew paraphrases over phrasing deltas ("pocket parks" vs "small green parks"; "per district" vs "per neighbourhood"). The prompt now distinguishes formulation differences (merge; keep the most concrete specifics) from intervention differences (refuse), with the observed false-merge pair (peak-hour buses vs night buses) as the negative example. Pre-registered A/B through the compiled function: English exactly unchanged (50/50 merges, 10/10 refusals); Hebrew wrong refusals 5→1, no new false merges; text-safety re-verified (scope inflation still 0/50).
8. **Synthesis attach is judged** (`f85df7f12`). The last geometry-only placement: attaching a statement to an existing synthesis now requires the semantic-equivalence judge's verdict `same` (verdict-cached; fail-closed). Motivated by four false merges that entered at cosine 0.899–0.944 — above every geometric gate.

> *The unifying law:* ***geometry proposes, judgement disposes — with no exceptions.*** *In English 3-small space, geometry was nearly sufficient and the judge-free shortcuts were safe by luck of the geometry. Hebrew 3-large compresses the space until distinct ideas are geometrically indistinguishable from paraphrases (twins span 0.73–0.97; cross-topic pairs reach 0.85). Every failure in this phase, in both languages, was a place where a cosine acted without judgement; every fix put a judge in front of one.*

4 Results

4.1 Score trajectories

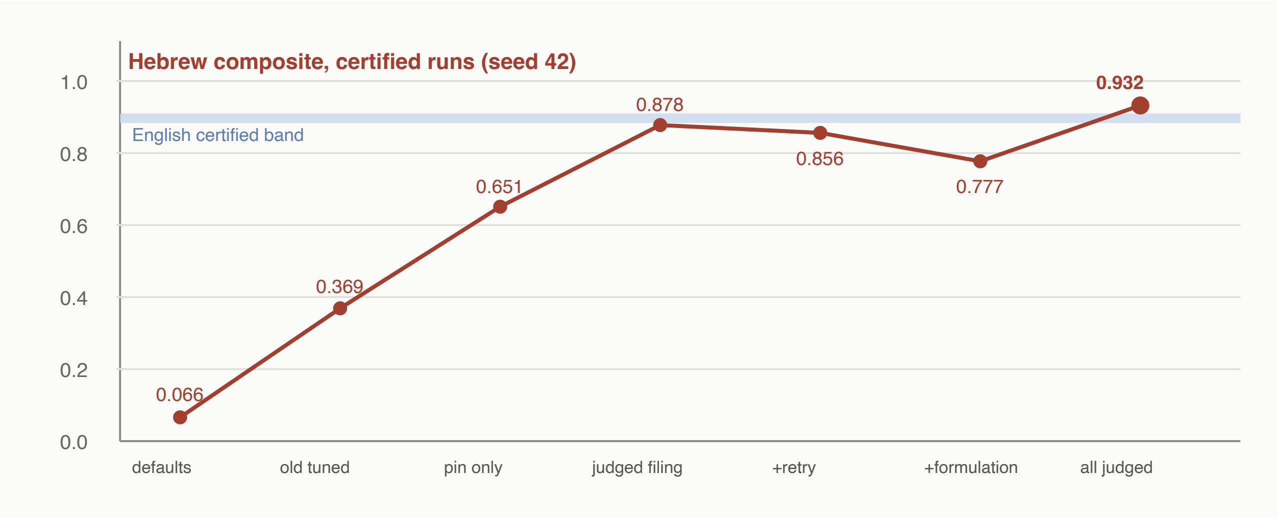

Figure 1 — the Hebrew arc. Shaded band: the certified English range (0.884–0.910). The two dips are runs whose per-mechanism changes were correct (verified offline) but whose composite was dominated by the then-unjudged attach path and theme-filing variance; see §4.3.

4.2 Certified runs of this phase

Run	Lang	Lever (cumulative)	Accuracy	F1 synth	False merges	F1 topic	Themes (true 10)
`en-seed42-memberjudge`	en	consolidation-on-contents (reverted)	0.865	0.990	0	0.678	10
`en-seed42-filingfix`	en	judged filing (contents + NONE-bias)	**0.902**	0.990	0	0.770	14
`he-seed42-large-perq`	he	3-large pin only	0.651	0.880	6	0.307	9
`he-seed42-large-judged`	he	+ judged topic attach, per-model bands	0.878	0.887	4	0.865	11
`he-seed42-large-recall`	he	+ spawn retry ×3, window 15	0.856	0.882	2	0.816	12
`he-seed42-large-formulation`	he	+ formulation ≠ intervention	0.777	0.857	6	0.658	11
`he-seed42-large-judgedattach`	he	+ judged synth attach	**0.932**	**0.947**	**0**	**0.910**	**10**
`en-seed42-alljudged`	en	regression: full all-judged build	0.878	0.990	0	0.710	14

Table 1 — certified runs, 2026-08-20 → 08-21. Full per-run diagnostics in `RESULTS.md` (Findings 15–19) and each run folder.

4.3 Reading the dips honestly

Runs 3 and 4 of the Hebrew arc scored *below* run 2 despite each shipping a correct, offline-verified improvement. The mechanism-level signals moved as predicted (spawn precision 0.915→0.953→…;

wrong refusals 5→1 offline), but the composite is 40% theme-filing, whose run-to-run variance (±0.05, driven by which heading titles the LLM happens to generate early) swamps a two-pair synthesis improvement. Meanwhile the then-unjudged attach path injected new false merges at cosine 0.899–0.944. Once that last path was judged, everything the earlier fixes had bought became visible at once: 0.932, synthesis precision 1.000. One attribution correction from the post-run audit: the winning run never actually exercised the new attach gate live (judged spawns formed the correct syntheses early, so no wrong attach was ever proposed) — the zero false merges belong to the ensemble; the gate stands as unit-tested insurance.

4.4 The English regression check

The all-judged build was re-run on English (no pin, 3-small, same seed): the synthesis layer is byte-identical to the certified best — F1 0.990, precision 1.000, zero false merges, 100/100 coverage — and the composite (0.878) differs from 0.902 only through topic filing drawing 0.710 from its known envelope (0.678–0.775 across certified runs; an untouched build drew 0.711 on seed 1234). The added judging cost English nothing in accuracy, and §5 shows it cost nothing measurable in time.

5 Effect on speed

All pipeline work is asynchronous background processing triggered on statement creation — no user-facing request waits on it. The relevant quantity is time-to-settle per statement: how long until a new statement is fully merged, filed, and all downstream recomputation is quiet. The benchmark harness measures exactly this (its per-statement wait has a fixed floor of ≈28 s from settle-detection windows; active pipeline time is the excess above the floor).

Run	Build	Mean / statement	Median	≈ Active pipeline time	Hit 45 s cap
`en-seed42-filingfix`	pre-judging changes	33.8 s	33.3 s	≈ 6 s	3/100
`en-seed42-alljudged`	all-judged	33.8 s	33.3 s	≈ 6 s	2/100
`he-seed42-large-judged`	judged filing/topics	35.4 s	32.6 s	≈ 7 s	11/100
`he-seed42-large-judgedattach`	all-judged	36.2 s	35.3 s	≈ 8 s	19/100

Table 2 — per-statement wall time under identical harness pacing. English is unchanged to the first decimal. Hebrew runs ≈2 s/statement slower than English, reflecting more LLM consultations per statement in the compressed geometry (more spawn attempts, more judge calls), and more statements still working at the harness's 45 s certification cap — a benchmark-pacing artifact, not a production concern, since production has no per-statement deadline.

Two design choices bound the added latency. The spawn retry only spends extra time *after a refusal*, which in well-separated (English) geometry is precisely when there is nothing to find. The attach judge and the cross-synth re-judge share a verdict cache keyed on text, so repeated encounters of the same pair are free.

6 Effect on cost

6.1 Call inventory per 100 statements (measured, final builds)

Call type	Tier	EN (alljudged)	HE (judgedattach)	Introduced / changed by
Embedding (statement)	embedding	≈100	≈100	unchanged (model per pin)
Synthesis spawn attempts	heavy	≈49–55	≈45–90	retry adds attempts only on refusal (cap 3)
Theme filing decisions	fast	≈99	≈97	existed; prompts now carry heading contents (+≈0.5–1k tokens/call)
Topic label generation	fast	28	17	unchanged
Consolidation sweep	fast	≈3–6	≈3–6	judge-once fingerprint *removed* ~144 calls/day per settled question
Attach semantic verdicts	fast, cached	0–10	0–10	new gate; fires only when geometry proposes an attach

Table 3 — LLM call inventory. Counts derived from certified run audit trails (placements, spawns, review queue) plus mechanism bounds; heavy-tier ranges reflect unlogged refused attempts, bounded by the retry cap.

6.2 What the changes add, and what they remove

- **Net LLM cost: ≈ +8–15% per statement, all in the fast tier.** The additions are: contents in filing prompts (larger fast-tier prompts), ~8–12 filing calls replacing what used to be free cosine attaches, occasional retry spawns, and the cached attach verdicts. The heavy tier — which dominates spend because synthesis calls generate full multi-paragraph proposals — is essentially unchanged in English and rises only with contested pairings in Hebrew.
- **One structural saving:** the consolidation sweep's judge-once fingerprint. Before it, every settled question re-asked the theme-merge judge every 10 minutes forever (≈144 calls/day/question doing nothing); now a theme set is judged once per actual change. On a deployment with many settled questions this saving alone plausibly exceeds everything the judging additions cost.
- **Embedding-model migration is a rounding error.** `text-embedding-3-large` costs ≈6.5× 3-small per token ($0.13 vs $0.02 per 1M tokens); at <100 tokens per statement, re-embedding even 100,000 statements costs on the order of one dollar. The +5 neighbour window adds ≈15 Firestore document reads per statement (≈$0.05 per 10,000 statements).
- **Caching does real work:** the attach gate reuses the reJudge sweep's verdict cache (keyed on text pair), and the model resolver caches per-question pins for 5 minutes, so per-question model resolution costs approximately one document read per question per 5 minutes per instance.

7 Threats to validity

- **One corpus, one shape.** A single civic question, exactly two paraphrases per idea, no off-topic or adversarial submissions. Every constant is calibrated against this geometry; the study's own README warns to re-measure before trusting the numbers in a different setting.
- **Composite variance.** Theme filing contributes ±0.05 run-to-run in both languages (heading titles are generated, and early titles steer later filings). 0.932 carries one run's filing fortune; the defensible claim is "reliably ≈0.9". A three-seed Hebrew sweep, as was done for English, is the natural next certification.
- **The winning run did not exercise the attach gate live** (no wrong attach was proposed). Its refusal behaviour is pinned by unit tests, not yet by a live incident.
- **Settle-cap censoring.** 2–19 statements per run hit the harness's 45 s wait cap while the pipeline was still working; late results are captured by the final settle pass but per-statement attribution near the cap is imprecise.

8 Deployment guidance

1. Nothing in this report is deployed. Deploy order: `npm run deploy:f:test` first, then production, as a separate explicit decision.
2. Migrating a named Hebrew question is one call — `reEmbedQuestion({questionId, embeddingModel: 'text-embedding-3-large'})` — which pins the model, writes the measured

bands for that geometry, and re-embeds every option. Statements arriving mid-migration already use the new model; the compatibility guard makes the transition degrade rather than corrupt.

3. Questions not migrated are byte-for-byte unaffected: the English regression run is the evidence.
4. The first live migration should be watched via `vectorSearch.incompatibleModel` (drop counts during transition) and `attach.judgeRefused` (the insurance gate's first live exercises).

9 Provenance

Findings 15–19 with full diagnostics: `RESULTS.md`. Run artifacts (inputs, results, audit trails, scores): `runs/<run-name>/`. Analysis instruments: `analysis/`. Key commits: `df336521c`, `d576f27f4` (filing), `5a8d37692` (per-question model), `e3a240562` (judged topic attach + bands), `bfff1c00b` (retry + window), `97e3b57dc` (formulation clause), `f85df7f12` (judged synth attach), `1b2263865` (English regression + attribution correction). All builds certified by `functions/lib` fingerprint; all tests green at each commit (636+ tests, 52 suites).